Part 3: Basic Usage

Agentic Coding Tools – A Comprehensive Guide

Instructor Name

University of South Carolina

January 23, 2026

Table of Contents

What Are Agentic Coding Tools?

AI-powered assistants that understand and interact with your development environment autonomously.

Unlike traditional code completion, these tools:

- Execute terminal commands directly
- Read and modify entire files
- Navigate your codebase structure
- Research solutions independently
- Commit changes to version control
- Reason about complex problems iteratively

Key Distinction

Traditional:
Single suggestion $\rightarrow$ Accept/Reject

Agentic:
Multi-step reasoning $\rightarrow$ Execute plan $\rightarrow$ Iterate

Categories of Agentic Tools

Terminal-Based

- Claude Code
- Aider
- GitHub Copilot CLI

Command-line focused, scriptable

IDE/Editor Plugins

- Cursor
- Windsurf
- VS Code extensions

Real-time as you work

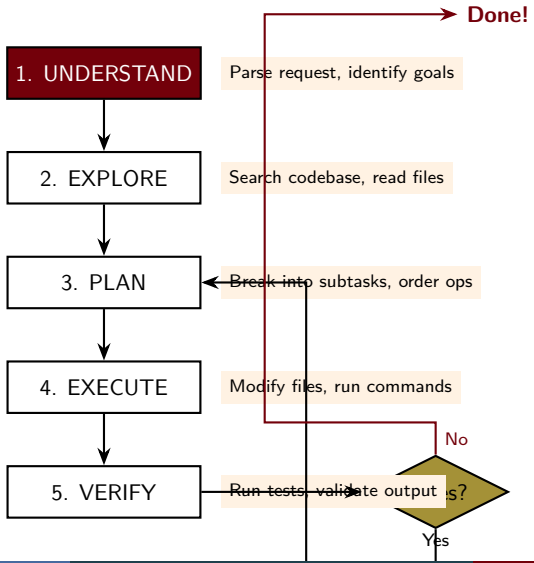
Web-Based

- Claude.ai
- ChatGPT
- GitHub Copilot Chat

Browser-based access

Terminal agents excel at: Automated workflows, multi-file refactoring, codebase analysis, CI/CD integration

The Agent Reasoning Loop

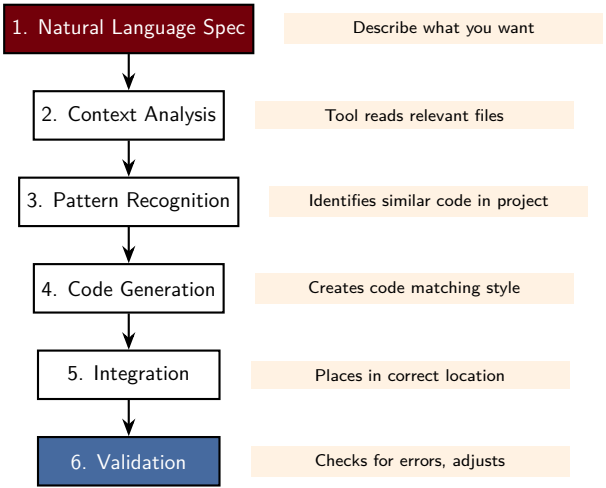


Core Capabilities at a Glance

Capability	Terminal	IDE	Description
Code Generation	✓✓	✓✓	Create files from scratch
Multi-File Editing	✓✓	✓	Modify multiple files at once
Codebase Search	✓✓	✓✓	Find patterns, understand arch.
Terminal Execution	✓✓	–	Run commands in real-time
Git Operations	✓✓	✓	Commit, branch, PR automation
Documentation	✓✓	✓	Generate docs, comments
Real-Time Feedback	–	✓✓	Immediate suggestions
Large Refactoring	✓✓	✓	Project-wide code changes

✓✓ = Excellent ✓ = Good – = Limited

Code Generation Fundamentals



Terminal Agent Example: Claude Code

```
# Start Claude Code with project context
claude code /workspace/api-project

# Request
> "Create a FastAPI endpoint that accepts POST requests
>  at /api/users with name and email, validates input
>  using Pydantic, stores in database, returns created
>  user with ID, includes error handling for duplicates"
```

Starting Claude Code

Claude Code will automatically:

1. Examine existing FastAPI route patterns
2. Check Pydantic model structure
3. Understand database schema from existing code
4. Generate endpoint matching project style
5. Add proper error handling and status codes
6. Run validation to ensure integration

Terminal Agent Example: Aider

```
# Install via pip
pip install aider-chat

# Start with specific files
aider src/main.py src/utils.py

# Interactive commands
aider> "Add JWT auth to login"
aider> "Refactor duplicated code"
aider> "Run tests, fix failures"
```

Aider Setup

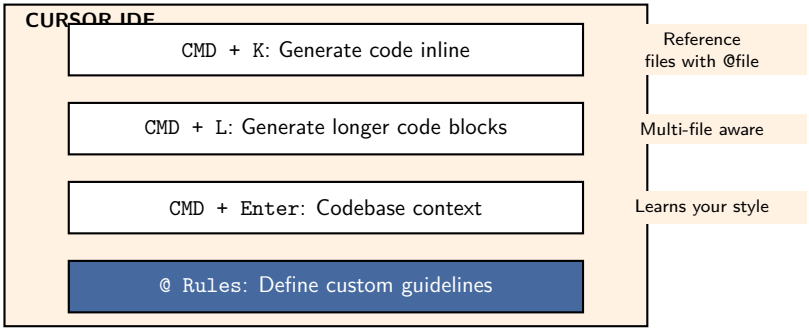
```
aider> "Update User model to
>       add email_verified field"

# Aider:
# - Reads User model file
# - Identifies all usages
# - Updates model, migrations
# - Updates related code
# - Runs tests to verify
```

Multi-File Refactoring

Key Aider Features:

- `/add src/file.py` – Mark files for editing
- `/commit 'message'` – Commit working changes
- Maintains context across multiple changes

IDE Agent Example: Cursor



Cursor learns your patterns: imports, error handling, component preferences

Multi-File Editing: The Power of Terminal Agents

Traditional Approach:

1. Update database schema
2. Create migration
3. Update ORM models
4. Update all queries
5. Update tests
6. Update documentation
7. Debug remaining issues

Hours of work...

With Terminal Agent:

```
aider> "Rename 'user_name' to 'username'
>       everywhere in the codebase.
>       Update migrations, models, queries,
>       tests, and API documentation."

# Agent output:
Found 47 occurrences in 6 files...
Updating files...
[1/6] migrations/001_rename.py
[2/6] models/user.py
[3/6] queries/user_queries.py
...
Running tests... All passing!
```

Minutes of work!

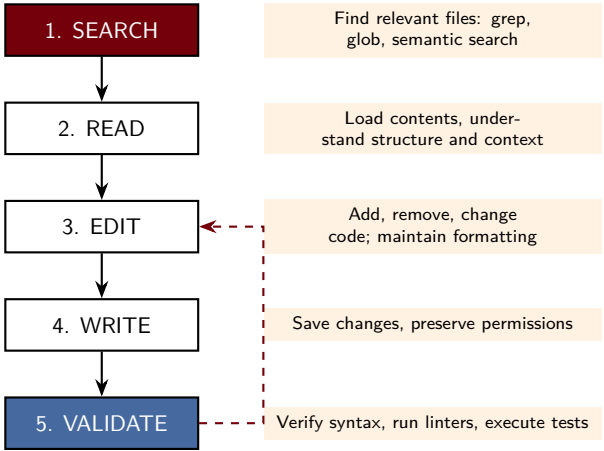
Choosing the Right Tool

Task	Best Tool	Why
Single function generation	Copilot, Cursor	Real-time, low friction
Component/module generation	Cursor, Windsurf	Multiple file context
Multi-file refactoring	Aider, Claude Code	Designed for this
Learning new framework	Cursor, Copilot	Immediate feedback
Understanding existing code	Copilot Chat, Claude Code	In-editor explanation
Batch automation	Claude Code, Terminal agents	Scriptable, repeatable
Architecture design	Claude Code, Windsurf Cascade	Reason about patterns

Recommendation

Use IDE plugins during development for rapid iteration, then terminal agents for larger refactoring tasks. The tools complement each other.

File Operations: Core Patterns



Creating Files & Project Structure

Single File Creation:

```
claude code /workspace

"Create .env.example with all
required environment variables
and helpful comments"
```

Result:

```
# .env.example
# Database Configuration
DATABASE_URL=postgresql://...
DATABASE_POOL_SIZE=10

# Authentication
JWT_SECRET=your-secret-key
JWT_EXPIRATION_HOURS=24
```

Full Project Structure:

```
my_service/
|-- src/
|   |-- __init__.py
|   |-- main.py
|   |-- config.py
|   |-- users/
|   |   |-- models.py
|   |   |-- routes.py
|   |   +-- services.py
|   +-- database/
|       +-- connection.py
|-- tests/
|   |-- conftest.py
|   +-- test_users.py
|-- docker/
|   +-- Dockerfile
|-- requirements.txt
+-- .env.example
```

Editing & Modifying Files

Before:

```
def login(email, password):
    user = db.query(User).filter(
        User.email == email
    ).first()
    if not user:
        return None
    if user.password == password:
        return user
    return None
```

After (with agent edit):

```
def login(email, password):
    user = db.query(User).filter(
        User.email == email
    ).first()
    if not user:
        raise ValueError("Invalid")
    if not verify_password(
        password, user.password_hash
    ):
        raise ValueError("Invalid")
    return user
```

Key Capabilities:

- Precise string replacement with context awareness
- Preserves indentation and formatting
- Maintains code style consistency
- Can handle multiple edits in single file atomically

Git Integration with File Operations

```
aider> "Add 'email_verified' field to User model.
>       Update migration, schema, tests, and API."

# Aider outputs:
Modified files:
  - models/user.py
  - database/migrations/20240115_add_email.py
  - schemas/user_schema.py
  - tests/test_user_model.py

Ready to commit? (y/n)
```

Automatic Staging for Related Changes

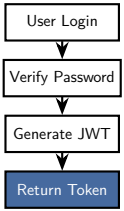
Benefits:

- Logical grouping of related changes
- Atomic commits that make sense

Documentation Generation from Code

```
claude code /workspace/project

"Generate comprehensive documentation
for the auth module in docs/AUTH.md.
Include:
- Overview of authentication system
- Flow diagrams (ASCII art)
- How to use each public function
- Security considerations
- Examples of common use cases"
```

Generated Output:



Plus code examples, security notes, and usage patterns

Inline Code Comments & Docstrings

Before:

```
def authenticate_user(email, password):
    user = db.query(User).filter(
        User.email == email
    ).first()
    if not user:
        return None
    if verify_password(
        password, user.password_hash
    ):
        return user
    return None
```

After (with docstrings):

```
def authenticate_user(
    email: str,
    password: str
) -> Optional[User]:
    """Authenticate user with
    email and password.

    Args:
        email: User's email.
        password: Plaintext password.

    Returns:
        User if successful, None
        otherwise.
    """
    # ... implementation
```

Request

"Add Google-style docstrings to all functions in models/user.py that don't have them."

Type Hints & Code Annotations

Before:

```
def get_user_by_email(email):
    return db.query(User).filter(
        User.email == email
    ).first()

def get_users_batch(user_ids):
    return db.query(User).filter(
        User.id.in_(user_ids)
    ).all()

def create_user(name, email, pwd):
    user = User(name=name, email=email)
    user.password_hash = hash(pwd)
    db.add(user)
    db.commit()
    return user
```

After:

```
from typing import Optional, List
from models import User

def get_user_by_email(
    email: str
) -> Optional[User]:
    return db.query(User).filter(
        User.email == email
    ).first()

def get_users_batch(
    user_ids: List[int]
) -> List[User]:
    return db.query(User).filter(
        User.id.in_(user_ids)
    ).all()

def create_user(
    name: str, email: str, pwd: str
) -> User:
    # ... typed implementation
```

Semantic Code Search

Text Search (Traditional)

```
grep "user_id" *.py
```

- Finds all lines containing text
- Lots of noise
- Manual filtering required

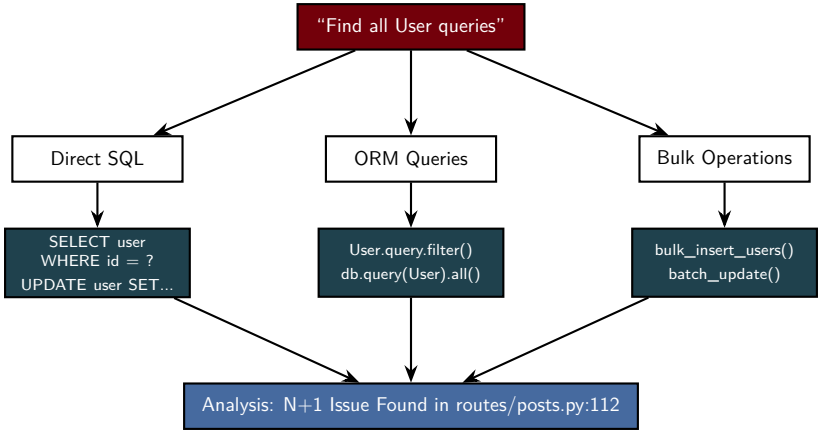
Semantic Search (Agentic)

"Find where user IDs are validated"

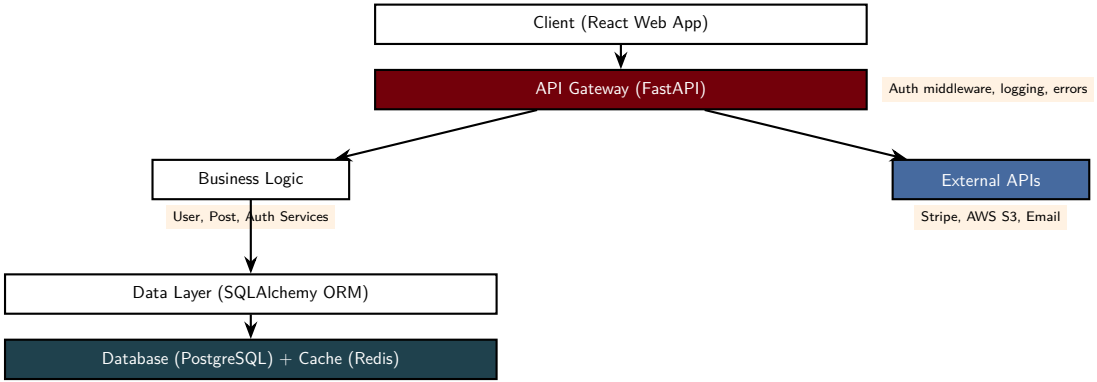
- Agent understands *validation* concept
- Returns validation functions
- Contextual results

Example Request: "Search the codebase for all database queries that touch the User table. Show direct queries, ORM queries, bulk operations, and identify any N+1 query problems."

Codebase Search Visualization



Architecture Understanding



Agent generates architecture diagrams, explains data flow, identifies patterns, and suggests improvements.

Dependency Mapping

Dependency Graph:

```
main.py
|-- models/
|   |-- user.py
|   |   +-- database/connection.py
|   +-- post.py
|       |-- database/connection.py
|       +-- models/user.py (CIRCULAR!)
|
|-- routes/
|   |-- user_routes.py
|   |   |-- services/user_service.py
|   |   +-- models/user.py
|   +-- post_routes.py
|
+-- middleware/
    |-- auth_middleware.py
    +-- error_handler.py
```

Impact Analysis:

```
aider> "I want to add 'avatar_url'
>       to User model. Show impact."

# Direct imports needing updates:
# - services/user_service.py
# - routes/user_routes.py
# - tests/test_user.py

# Database impacts:
# - Need migration for new field
# - Update fixtures in tests

# API impacts:
# - User response schema
# - Documentation
```

Benefit: Understand refactoring impact before making changes!

Finding Patterns & Antipatterns

Antipatterns Found:

N+1 Query Problem

```python
# BAD: routes/posts.py:112
posts = db.query(Post).all()
for post in posts:
    author = db.query(User)...

# GOOD: Use eager loading
posts = db.query(Post).options(
    joinedload(Post.user)
).all()
```

Bare Except

```python
# BAD: Catches everything!
except:
    return None

# GOOD: Specific exceptions
except RedisError:
    return None
```

Code Duplication Found:

Type 1: Copy-Paste

- Email validation in 2 files
- Extract to `validators.py`

Type 2: Similar Logic

- Error formatting in 3 middlewares
- Centralize in `error_handler.py`

Type 3: Validation

- Password validation in 3 places
- Extract to single validator

Web Search & URL Fetching

Direct Web Search:

```
claude code /workspace

"Search the web for:
1. JWT token best practices 2024
2. FastAPI security vulnerabilities
3. PostgreSQL vs MongoDB benchmarks
4. Python async optimization"

# Returns:
# - Current articles
# - Stack Overflow solutions
# - GitHub implementations
# - Performance benchmarks
```

URL Content Analysis:

```
"Visit fastapi.tiangolo.com/
advanced/middleware/ and tell me:
1. How middleware works
2. Key concepts
3. Code examples
4. Framework differences"

# Agent:
# - Fetches page
# - Extracts content
# - Analyzes examples
# - Summarizes concepts
```

Research-Driven Development

"We're considering switching from SQLAlchemy to SQLModel. Search for comparisons, adoption rates, known issues, and give me a recommendation."

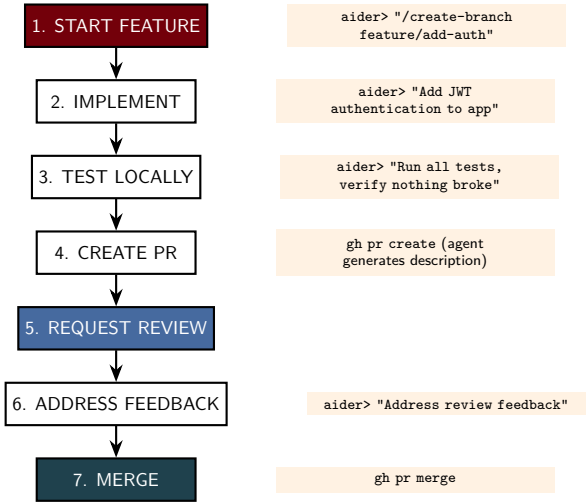
Chaining Research Tasks

```
claude code /workspace

"Evaluate migrating from JWT to OAuth2. Please:
1. Search for security best practices comparing them
2. Fetch OAuth2 RFC and RFC 6750 standards
3. Analyze our current JWT implementation
4. Find 3 well-regarded OAuth2 open-source projects
5. Compare implementation complexity vs security
6. Estimate effort to migrate our 50+ endpoints
7. Make a recommendation with justification"
```

Result: Comprehensive analysis that would take a human *days* to compile, completed in minutes.

Power of chaining: Agent handles investigation upfront, then applies findings directly to your code.

Git Workflow with Agents



Commit Management

Atomic Commits:

```
aider> "Create 4 separate atomic commits:
>       1. Password hashing to Argon2
>       2. Email verification
>       3. Token generation refactor
>       4. Tests and documentation"

# Results:
feat(auth): migrate to Argon2
- Replace bcrypt for better security
- Update password validation

feat(auth): add email verification
- Require verification after signup
- Send emails via SMTP

refactor(auth): simplify token gen
- Extract token generation logic
- Improve error messages

test(auth): add tests and docs
- 12 new test cases
- Update API documentation
```

Conventional Commits:

Format: `type(scope): subject`

`feat`	New feature
`fix`	Bug fix
`refactor`	Code refactoring
`perf`	Performance
`test`	Adding tests
`docs`	Documentation
`chore`	Build, deps

Agents automatically:

- Analyze staged changes
- Generate meaningful messages
- Follow project conventions

Pull Request Creation

```
# Pull Request: Refactor Authentication System

## Description
Refactors auth system to improve security, maintainability,
and testability. All auth logic moved to dedicated service.

## Changes Made
- Created services/auth_service.py with centralized auth logic
- Migrated from bcrypt to Argon2 for password hashing
- Added email verification workflow

## Testing
- Added 15 new unit tests for auth service
- All existing tests still pass (127/127)

## Deployment Notes
- Requires new env vars: ARGON2_MEMORY, ARGON2_TIME
- Database migration required
```

Merge Conflict Resolution

Conflict Analysis:

```
File: src/services/user_service.py

Conflict 1 (Line 45):
Main branch:
  def get_user(user_id: int) -> User:

Feature branch:
  def get_user(user_id: int)
      -> Optional[User]:

Resolution: Use feature branch
(adds Optional type hint - improvement)

Conflict 2 (Line 67):
Main: Added caching decorator
Feature: Added logging decorator

Resolution: Keep both - combinable
  @cache_result
  @log_execution
  def search_users(query: str):
```

Request:

```
claude code /workspace

"Our feature branch has
conflicts with main. Please:
1. Identify conflicting
   files
2. Analyze root causes
3. Propose resolutions
4. Implement resolutions
5. Test to ensure nothing
   broke"
```

Agent provides:

- Side-by-side comparison
- Intelligent merge proposals

Key Takeaways

Terminal Agents

Claude Code, Aider, GitHub Copilot CLI

- Chain operations automatically
- Best for multi-file refactoring
- Scriptable and repeatable
- CI/CD integration

IDE Plugins

Cursor, Windsurf, VS Code Copilot

- Immediate suggestions
- Less context-switching
- Great for learning
- Rapid prototyping

Web Integration

- Search for solutions
- Fetch documentation
- Stay current with best practices

Git Integration

- Atomic commits
- Meaningful messages
- Branch management
- PR automation

Next Steps

1. **Choose your primary tool**
   - Recommend: Start with Cursor or Claude Code
2. **Practice simple tasks first**
   - File creation, reading, editing
3. **Graduate to multi-file operations**
   - Refactoring, testing
4. **Integrate into your workflow gradually**
5. **Build custom workflows and automation**

What's Next

Part 4 will cover: Advanced techniques, custom workflows, performance optimization, debugging with agentic tools, and building your own agentic workflows.

Quick Reference

Terminal Agent Commands:

- `claude code /path/to/project`
- `aider src/file1.py src/file2.py`
- `gh copilot suggest "..."`
- `gh copilot explain "..."`

Common Task Prompts:

- "Create a function that [desc]"
- "Refactor [file] to [improvement]"
- "Write tests for [module]"
- "Help me debug [error]"
- "Generate documentation for [module]"

Resources: Official documentation, YouTube tutorials, GitHub examples, community forums

End of Part 3

Basic Usage

Questions?